

# GABRIEL POPA

Senior Software Engineer - Angular • Python (Django/ FastAPI) • AWS

Cluj, Romania | +40 764309991 | gabrielpdev@outlook.com | <https://www.linkedin.com/in/gabriel-popa-12890b414/>

## ABOUT ME

---

Senior Software Engineer with 10+ years of experience building business web platforms across **Python 2.7–3.12**, **AngularJS–Angular 18**, **Django 1.x–4.x**, **FastAPI**, **Flask**, **TypeScript 3.x–5.x**, **PostgreSQL**, **Redis**, **Celery**, **RabbitMQ**, **Docker**, and **AWS**, with strongest specialization in API-driven backend delivery and Angular frontends for workflow-heavy systems, plus a practical record of production issue investigation, bug fixing, legacy modernization, runtime and framework upgrades, secure integration work, performance tuning, and shipping maintainable features that balance speed, reliability, and long-term supportability.

## SKILLS

---

- **Frontend: AngularJS, Angular 2–18, TypeScript**, JavaScript, RxJS, Angular Material, Reactive Forms, route guards, lazy loading, reusable component architecture, HTML5, CSS3, SCSS, responsive UI, client-side validation
- **Backend: Python 2.7–3.12, Django 1.x–4.x, FastAPI, Flask**, REST APIs, background jobs, service integration, authentication, authorization, validation, file processing, admin workflows, API versioning, asynchronous processing
- **Database / Data & Messaging: PostgreSQL**, MySQL, SQLite, **Redis**, **Celery**, **RabbitMQ**, query optimization, indexing, caching strategies, reporting workflows, ETL support, schema changes, data integrity controls
- **Cloud & DevOps: AWS**, Docker, Linux, Nginx, GitHub Actions, Jenkins, CI/CD pipelines, deployment automation, environment configuration, release validation, containerized delivery
- **Observability / Quality / Security:** Sentry, structured logging, OpenTelemetry basics, Prometheus/Grafana basics, correlation IDs, Pytest, Jest, Jasmine, Karma, Cypress, Playwright, unit testing, integration testing, end-to-end testing, OWASP security practices, rate limiting, audit logging

## PROFESSIONAL EXPERIENCE

---

### Senior Software Engineer

Soft Build | Bucharest, Romania (Jan 2024 – May 2026)

- Led modernization of a workflow-heavy platform with **Python 3.11–3.12**, **FastAPI**, and **Angular 16–17**, turning fragmented reporting and administration flows into cleaner modules that were easier to extend safely.
- Reworked brittle dashboard screens into reusable **Angular** feature areas with stronger typing, route-based boundaries, and shared form patterns, which improved delivery speed and reduced repeated **UI defects** across teams.
- Investigated a recurring **production bug** where multi-step submission flows lost state after validation failures, then stabilized the behavior through better form orchestration, clearer **API contracts**, and safer client recovery handling.
- Migrated older **Python** service code into a more maintainable **FastAPI** structure with explicit schemas, dependency-based validation, and cleaner separation of controllers and domain logic, which lowered regression risk during releases.
- Solved slow reporting screens by profiling **PostgreSQL** execution paths, removing unnecessary joins, and introducing selective **Redis caching**, which improved high-usage query response times by **34%** without sacrificing data accuracy.
- Built asynchronous export and synchronization processing with **Celery** and **Redis** so long-running operations no longer blocked request threads, giving users clearer progress visibility and reducing failed requests during peak usage.
- Resolved a data consistency issue where concurrent edits occasionally overwrote recent changes by implementing **optimistic locking**, safer retry rules, and conflict-aware responses that prevented silent record corruption.

- Strengthened security on sensitive operational endpoints by tightening request validation, improving token handling, adding practical **rate limiting**, and expanding **audit logging** so support teams could investigate changes more confidently.
- Introduced **structured logging**, correlation IDs, and **Sentry** coverage around critical API and job paths, which shortened issue investigation time and gave the team **38%** earlier visibility into tenant-specific failures.
- Handled a staged **runtime upgrade** and dependency update for older Python services that had accumulated compatibility debt, using targeted refactors and broader test coverage to move onto a more supportable baseline safely.
- Delivered new product features across both **backend** and **frontend** layers, moving fluidly between **Angular forms**, **API validation**, **database changes**, and worker logic whenever release timelines required broader cross-stack ownership.
- Provided practical **mentoring** during code reviews and incident follow-ups by explaining debugging steps, upgrade risks, and safer refactoring choices, which improved team confidence on fragile modules without slowing delivery.

## Software Engineer

Next Apps | Ghent, Belgium (Nov2020 – Nov 2023)

- Built customer-facing product workflows with **Python 3.8–3.11**, **Django**, **FastAPI**, and **Angular 9–16**, helping the team deliver validation-heavy screens and backend features with more predictable behavior.
- Modernized selected frontend areas from older **Angular** patterns into cleaner **Reactive Forms** and shared UI building blocks, which reduced duplicated validation logic and made new screens easier to implement consistently.
- Investigated inconsistent dashboard totals reported by support teams, then corrected **Redis invalidation** and backend update sequencing so cached values reflected the true persistence state more reliably.
- Developed reusable form, table, modal, and notification components that improved consistency across multiple modules and reduced copy-paste implementation effort by **32%** during ongoing feature work.
- Supported **third-party integrations** where unstable payloads and intermittent upstream failures caused user-facing issues, then added normalization, retry handling, and better logging to make those failures easier to diagnose.
- Migrated selected backend modules from older **Django** views into cleaner **API-oriented service boundaries**, which improved validation clarity and created a more maintainable path for future **FastAPI** adoption where appropriate.
- Resolved duplicate processing in asynchronous jobs by reviewing **Celery retry behavior**, cleanup timing, and idempotency assumptions, which improved worker reliability and reduced support-side confusion around failed tasks.
- Improved production visibility by adding **Sentry**, structured exception mapping, and clearer backend error responses, which helped the team identify and prioritize real failures **40%** faster before they spread.
- Strengthened **test coverage** around permissions, validation logic, and workflow-heavy APIs with **Pytest** and frontend checks, making refactors safer in areas that had previously produced recurring regressions.
- Helped standardize **Docker-based** local environments and deployment configuration so production-like issues could be reproduced earlier, which reduced onboarding friction and improved release predictability.
- Worked closely across **frontend**, **backend**, and **release tasks** as priorities shifted, giving the team flexible support on **bug fixing**, **feature delivery**, and **modernization work** instead of staying limited to one layer.

## Software Developer

Datamart | Stockholm, Sweden (Feb 2016 – Sep 2020)

- Built internal business applications with **Python 3.5–3.8**, **Django**, **Flask**, and **AngularJS** to **early Angular**, helping evolve older reporting workflows into more maintainable **full-stack solutions**.
- Improved admin usability by refining search, filtering, and export behavior in screens that handled large datasets, which made day-to-day operational work smoother for teams relying on reporting accuracy.
- Solved a persistent reporting defect caused by **timezone mismatches** between browser input, API serialization, and database storage by standardizing date handling rules across the full request lifecycle.
- Modernized selected UI areas away from template-heavy patterns into more structured **Angular components**, which reduced fragile view logic and made repeated admin workflows easier to maintain.
- Optimized slow **PostgreSQL** queries by reviewing execution plans, adding missing indexes, and reducing unnecessary reads, which improved reporting responsiveness by **35%** on high-use operational screens.

- Moved long-running imports and large export generation into **background processing** so synchronous requests stopped timing out, reducing disruption during busy periods and improving process stability by **25%**.
- Supported a gradual backend refactor from tightly coupled modules into smaller **service-oriented** pieces, allowing feature work and maintenance fixes to continue without forcing a risky full rewrite.
- Improved deployment safety by assisting with **Jenkins pipelines**, validation scripts, and environment checks, which reduced packaging and release mistakes while making deployments more repeatable.
- Added clearer logging and targeted tests around validation and transformation paths, which made **production bugs** easier to reproduce and lowered regression risk during maintenance and small enhancements.
- Contributed across **frontend, backend, data, and release activities** depending on need, showing flexible ownership on **bug resolution, incremental modernization**, and practical feature support in a mixed legacy environment.

## Junior Software Developer

Berg Software | Timișoara, Romania (Sep2014 – Mar 2016)

- Contributed to a legacy business application built with **Python 2.7–3.5, Django**, JavaScript, and early **AngularJS**, helping deliver safe changes in tightly coupled production modules.
- Maintained server-rendered pages, backend forms, and validation logic where stable behavior mattered more than aggressive rewrites, keeping day-to-day workflows dependable during ongoing minor updates.
- Fixed a recurring **login problem** by aligning cookie settings, timeout behavior, and request handling between browser and backend flows, which improved sign-in stability for regular users.
- Added small **AngularJS-based** interactive behaviors to selected pages for inline actions and dynamic form feedback while keeping the implementation compatible with the existing template-driven structure.
- Improved form handling in administrative screens by tightening validation, cleaning inconsistent error messages, and correcting edge-case inputs, which reduced failed submissions by **24%** for common tasks.
- Assisted with **defect investigation** by reproducing issues from support notes and logs, then preparing focused low-risk fixes that senior engineers could review and release with confidence.
- Helped improve performance on slower pages by reducing unnecessary queries and simplifying backend data preparation, which made frequently used screens feel more responsive and cut support complaints by **18%**.
- Supported senior engineers during **framework** and **dependency update** work by checking compatibility problems, fixing deprecated code paths, and verifying affected screens in fragile areas.
- Built reusable helper functions, added basic tests, and improved logging in selected workflows, which strengthened troubleshooting quality and gave the team a more reliable base for future changes.

## EDUCATION

---

### Bachelor's Degree in Computer Science

University of Bucharest | (2010 – 2014)

- Focused on practical software engineering fundamentals that translated directly into building reliable web systems and data-driven applications.

## CERTIFICATIONS

---

### AWS Certified Developer – Associate — 2023

- Demonstrated practical capability in developing and maintaining **AWS-based** applications, with focus on deployment, service integration, monitoring, and secure cloud operations.

### PCAP – Certified Associate in Python Programming — 2022

- Demonstrated strong foundation in **Python** programming, including core language features, modular design, debugging, and problem-solving for backend-oriented applications.

### Scrum Fundamentals Certified (SFC) — 2021

- Demonstrated working knowledge of agile principles, sprint-based execution, team collaboration, and structured delivery practices within software development environments.